# A Bible of Scientific Machine Learning

*Dynamical Systems, Operators, and the Frontier of Physics-Aware Deep Learning*

A reference manual covering classical dynamics, PINNs, neural operators, Koopman/SINDy methods, equivariant networks, generative surrogates, uncertainty quantification, and 2024–2026 frontiers.

Compiled for **Julian C.**

PhD researcher — Astrophysics & Machine Learning

May 1, 2026

*"All models are wrong, but some are useful."*
*— George E. P. Box*

*"It is the theory which decides what we can observe."*
*— Albert Einstein*

*"Mathematics is the art of giving the same name to different things."*
*— Henri Poincaré*

# Preface

Scientific Machine Learning (SciML) is the discipline that fuses the *inductive bias of physics* with the *expressive flexibility of modern deep learning*. Where classical numerical analysis treats the governing equations of nature as the primary object and data as a secondary product, and where pure machine learning treats data as primary and equations as optional, SciML insists on the dialogue between the two. The result is a family of methods — physics-informed neural networks, neural operators, Koopman embeddings, universal differential equations, equivariant networks, diffusion-based surrogates, foundation models for physics — that are reshaping how scientists model complex dynamical systems.

## Who this is for

This document is written for advanced graduate students and researchers working at the intersection of dynamical systems, numerical analysis, and deep learning. It assumes familiarity with multivariable calculus, linear algebra, ODE/PDE theory, and the basic apparatus of supervised deep learning (backpropagation, optimisation, common architectures).

## How to read it

The book is structured as a reference. Part I (Chapters 1–2) lays the mathematical foundations of dynamical systems and classical numerics. Part II (Chapters 3–7) covers the core SciML toolkit. Part III (Chapters 8–11) addresses geometric, generative, and probabilistic extensions. Part IV (Chapters 12–13) surveys software, benchmarks, and emerging trends through the 2024–2026 horizon. The appendix (Chapter 14) collects functional analysis, automatic differentiation, and stochastic calculus prerequisites.

Theorems, definitions, recipes, pitfalls, and insights are colour-coded throughout to help the reader scan rapidly:

**Recipe: Pseudocode and step-by-step procedures**

Practical workflows live in green recipe boxes.

**Insight: Conceptual remarks**

Purple boxes flag conceptual subtleties or unifying perspectives.

**Pitfall: Things that go wrong**

Red boxes highlight failure modes that have caught practitioners.

## Notation

We follow standard conventions: bold lowercase for vectors ($\boldsymbol{x} \in \mathbb{R}^d$), bold uppercase for matrices ($\boldsymbol{A}$), calligraphic for operators and function spaces ($\mathcal{L}, \mathcal{G}, \mathcal{H}$). Time is denoted $t$, spatial coordinates $\boldsymbol{x}$ or $\boldsymbol{\xi}$. Neural network parameters are $\theta$. Probability density functions are $p(\cdot)$. Expectations are $\mathbb{E}[\cdot]$. The differential operator is $\partial$, the gradient $\nabla$, the Laplacian $\Delta$, and the divergence $\nabla\cdot$.

## Caveats

SciML moves fast. Where possible we highlight *stable principles* rather than transient implementations. Citations point to canonical references; the field's pace means new results may already supersede specific benchmarks by the time of reading.

# Contents

**Preface** **v**

**1 Foundations of Dynamical Systems** **1**
1.1 What is a dynamical system? . . . 1
1.2 Linear systems and stability . . . 1
1.3 Nonlinear phenomena . . . 2
1.4 Hamiltonian and Lagrangian mechanics . . . 2
1.5 Partial differential equations . . . 2
1.6 Function spaces and the operator picture . . . 3
1.7 Stochastic dynamics . . . 3
1.8 Inverse problems . . . 3
1.9 Summary . . . 3

**2 Classical Numerical Methods** **4**
2.1 Time integration of ODEs . . . 4
2.1.1 Explicit Runge–Kutta . . . 4
2.1.2 Implicit and stiff solvers . . . 4
2.1.3 Symplectic integrators . . . 5
2.2 Spatial discretisation of PDEs . . . 5
2.2.1 Finite differences (FD) . . . 5
2.2.2 Finite volumes (FV) . . . 5
2.2.3 Finite elements (FE) . . . 5
2.2.4 Spectral methods . . . 5
2.2.5 Mesh-free / particle methods . . . 5
2.3 Linear algebra and Krylov methods . . . 5
2.4 Adjoint and sensitivity analysis . . . 6
2.5 Stability, consistency, convergence . . . 6
2.6 Reduced-order models (ROMs) . . . 6
2.7 Summary . . . 6

**3 Neural ODEs, CDEs, and SDEs** **7**
3.1 From ResNets to Neural ODEs . . . 7
3.1.1 Adjoint method . . . 7
3.2 Variants and architectures . . . 7
3.3 Hamiltonian and Lagrangian neural networks . . . 8

3.4 ODE-based deep generative models . . . 8
3.4.1 Continuous normalising flows (CNF) . . . 8
3.4.2 FFJORD and Hutchinson trace estimators . . . 8
3.4.3 Flow matching and stochastic interpolants . . . 8
3.5 Practical training tips . . . 8
3.6 Where neural ODEs shine and where they struggle . . . 9

**4 Physics-Informed Neural Networks 10**
4.1 Formulation . . . 10
4.2 Why it works (when it works) . . . 10
4.3 Variants . . . 11
4.4 Failure modes . . . 11
4.4.1 Mitigations . . . 11
4.5 Inverse problems with PINNs . . . 12
4.6 Theoretical guarantees . . . 12
4.7 When to choose a PINN over alternatives . . . 12
4.8 Code skeleton (JAX) . . . 12
4.9 Summary . . . 13

**5 Neural Operators 14**
5.1 Solution operators . . . 14
5.2 DeepONet . . . 14
5.3 Fourier Neural Operator (FNO) . . . 14
5.3.1 Variants of FNO . . . 15
5.4 Graph Neural Operators . . . 15
5.5 Transformer-based operators . . . 15
5.6 Discretisation invariance: theory . . . 15
5.7 Training data . . . 16
5.8 Beyond pure data: physics-informed operators . . . 16
5.9 Limitations and open problems . . . 16
5.10 Choosing an operator architecture . . . 16

**6 Operator-Theoretic Learning: Koopman, DMD, SINDy 17**
6.1 The Koopman operator . . . 17
6.1.1 Dynamic Mode Decomposition (DMD) . . . 17
6.1.2 Extended DMD (eDMD) . . . 17
6.1.3 Deep Koopman . . . 17
6.2 Sparse identification of nonlinear dynamics (SINDy) . . . 18
6.2.1 Variants . . . 18
6.2.2 Pitfalls . . . 18
6.3 Symbolic regression . . . 18
6.4 Reservoir computing and echo state networks . . . 19
6.5 Recipe: pick the right operator-theoretic tool . . . 19
6.6 Connections to neural operators . . . 19

**7 Hybrid Models and Universal Differential Equations 20**
7.1 Universal differential equations . . . 20
7.1.1 Why hybrid beats both extremes . . . 20
7.1.2 Discovery loop . . . 20
7.2 Closure modelling . . . 21
7.3 Differentiable simulators . . . 21

7.4 Kolmogorov–Arnold Networks (KAN) . . . 21
7.5 Operator + physics hybrids . . . 21
7.6 Equation-discovery pipelines . . . 22
7.7 Summary . . . 22

**8 Geometric and Equivariant Deep Learning 23**
8.1 Why symmetry matters . . . 23
8.2 Group-equivariant CNNs . . . 23
8.3 $E(3)$ and $SE(3)$ equivariance . . . 23
8.4 N-body dynamics . . . 24
8.5 Hamiltonian / Lagrangian + equivariance . . . 24
8.6 Mesh-based and graph-based PDE surrogates . . . 24
8.7 Open challenges . . . 24
8.8 Recipe: choosing an equivariant model . . . 25

**9 Generative Models for Dynamical Systems 26**
9.1 Why generative? . . . 26
9.2 Diffusion models . . . 26
9.2.1 Score-based formulation . . . 26
9.2.2 Diffusion for PDEs . . . 26
9.2.3 Conditional / guided diffusion . . . 27
9.3 Flow matching and stochastic interpolants . . . 27
9.4 Generative ensembles for weather and climate . . . 27
9.5 GANs and energy-based models in SciML . . . 27
9.6 Practical considerations . . . 27

**10 Uncertainty Quantification 29**
10.1 Sources of uncertainty . . . 29
10.2 Bayesian deep learning . . . 29
10.2.1 Bayesian neural networks (BNNs) . . . 29
10.2.2 Bayesian PINNs . . . 29
10.3 Deep ensembles . . . 29
10.4 Gaussian processes . . . 30
10.5 Conformal prediction . . . 30
10.6 Probabilistic operators and probabilistic PINNs . . . 30
10.7 Calibration and evaluation . . . 30
10.8 Active learning and experimental design . . . 31
10.9 Practical guide . . . 31

**11 Multiscale and Multi-Physics SciML 32**
11.1 Classical multiscale methods . . . 32
11.2 Where ML fits . . . 32
11.3 Foundation models for multi-physics . . . 32
11.4 Coupled physics: examples . . . 33
11.5 Scale invariance and dimensional analysis . . . 33
11.6 Practical advice . . . 33

**12 Software, Datasets, and Benchmarks 34**
12.1 Software ecosystems . . . 34
12.1.1 Julia . . . 34
12.1.2 Python / JAX . . . 34
12.1.3 Other . . . 34

12.2 Datasets and benchmarks . . . 34
12.2.1 PDE benchmarks . . . 34
12.2.2 Molecular / materials . . . 35
12.2.3 Climate / weather . . . 35
12.2.4 Astrophysics . . . 35
12.3 Reproducibility checklist . . . 35
12.4 Hardware considerations . . . 35

**13 Emerging Trends (2024–2026) 36**
13.1 Foundation models for physics . . . 36
13.2 Diffusion and flow-based generative simulators . . . 37
13.3 Transformer operators . . . 37
13.4 Rethinking PINNs . . . 37
13.5 Equation discovery at scale . . . 37
13.6 AI for science programmes . . . 37
13.7 Quantum-enhanced SciML . . . 38
13.8 Active learning, autonomous experimentation, and AI agents . . . 38
13.9 Reinforcement learning for control of dynamical systems . . . 38
13.10 Where to bet next . . . 38

**14 Appendices 39**
Appendix A: Automatic Differentiation . . . 39
Appendix B: Functional Analysis Cheat Sheet . . . 39
Appendix C: Stochastic Calculus . . . 39
Appendix D: Reading List . . . 40
Appendix E: Glossary . . . 40
Appendix F: Method Taxonomy . . . 41

**References & Further Reading 42**

# 1

# Foundations of Dynamical Systems

*The whole of science is nothing more than a refinement of everyday thinking.*

Albert Einstein

## 1.1 What is a dynamical system?

A *dynamical system* is a triple $(M, T, \Phi)$ where $M$ is a state space (a manifold), $T$ is a time set (typically $\mathbb{R}$ or $\mathbb{Z}$), and $\Phi : T \times M \to M$ is an evolution map satisfying the semi-group axioms

$$\Phi(0, \boldsymbol{x}) = \boldsymbol{x}, \tag{1.1}$$

$$\Phi(t + s, \boldsymbol{x}) = \Phi(t, \Phi(s, \boldsymbol{x})). \tag{1.2}$$

Continuous-time systems are usually specified by an ODE

$$\dot{\boldsymbol{x}} = \boldsymbol{f}(\boldsymbol{x}, t; \mu), \qquad \boldsymbol{x}(0) = \boldsymbol{x}_0, \tag{1.3}$$

with parameter $\mu \in \mathbb{R}^p$. Discrete systems are given by a map $\boldsymbol{x}_{k+1} = \boldsymbol{F}(\boldsymbol{x}_k)$. Spatially extended systems are modelled by PDEs and become infinite-dimensional dynamical systems on function spaces.

**Definition 1.1: Flow and orbit**

The *flow* of Equation (1.3) is the family of maps $\{\Phi^t\}_{t\in\mathbb{R}}$ with $\Phi^t(\boldsymbol{x}_0) = \boldsymbol{x}(t)$. The *orbit* through $\boldsymbol{x}_0$ is the set $\gamma(\boldsymbol{x}_0) = \{\Phi^t(\boldsymbol{x}_0) : t \in \mathbb{R}\}$.

## 1.2 Linear systems and stability

For the linear ODE $\dot{\boldsymbol{x}} = \boldsymbol{A}\boldsymbol{x}$ the solution is $\boldsymbol{x}(t) = e^{t\boldsymbol{A}}\boldsymbol{x}_0$. The eigenvalues of $\boldsymbol{A}$ classify the origin: stable node, unstable node, saddle, focus, centre. For nonlinear systems, the *Hartman–Grobman theorem* guarantees that, near a hyperbolic fixed point, the flow is topologically conjugate to its linearisation.

**Theorem 1.1: Lyapunov stability**

Let $\boldsymbol{x}^\star$ be an equilibrium of Equation (1.3). If there exists a continuously differentiable function $V : \Omega \to \mathbb{R}$ on a neighbourhood $\Omega \ni \boldsymbol{x}^\star$ with $V(\boldsymbol{x}^\star) = 0$, $V(\boldsymbol{x}) > 0$ for $\boldsymbol{x} \neq \boldsymbol{x}^\star$,

and $\dot{V}(\boldsymbol{x}) \leq 0$ on $\Omega$, then $\boldsymbol{x}^\star$ is Lyapunov stable. If $\dot{V} < 0$ strictly, it is asymptotically stable.

## 1.3 Nonlinear phenomena

Nonlinearity introduces phenomena absent from linear theory:

- **Limit cycles:** isolated periodic orbits (Van der Pol).
- **Bifurcations:** qualitative changes of phase portraits as parameters vary — saddle-node, transcritical, pitchfork, Hopf, period doubling, homoclinic.
- **Chaos:** sensitive dependence on initial conditions; the maximal Lyapunov exponent $\lambda_{\max} = \lim_{t\to\infty} \frac{1}{t} \log \|\delta\boldsymbol{x}(t)\|/\|\delta\boldsymbol{x}(0)\|$ is positive.
- **Strange attractors:** fractal invariant sets attracting nearby orbits (Lorenz, Rössler, Chua).

**Insight: Why dynamics matters for ML**

Many learning algorithms are themselves dynamical systems: gradient descent is an Euler discretisation of gradient flow $\dot{\theta} = -\nabla L(\theta)$; momentum methods correspond to damped Hamiltonian dynamics; sampling algorithms (Langevin, HMC) are stochastic flows on parameter space. The mathematical machinery in this chapter is therefore both the *object* and the *tool* of SciML.

## 1.4 Hamiltonian and Lagrangian mechanics

Many physical systems possess a symplectic structure. Given a configuration manifold $Q$ with coordinates $\boldsymbol{q}$ and conjugate momenta $\boldsymbol{p}$, the *Hamiltonian* $H(\boldsymbol{q}, \boldsymbol{p}, t)$ generates flow via

$$\dot{\boldsymbol{q}} = \frac{\partial H}{\partial \boldsymbol{p}}, \qquad \dot{\boldsymbol{p}} = -\frac{\partial H}{\partial \boldsymbol{q}}. \tag{1.4}$$

The 2-form $\omega = \mathrm{d}\boldsymbol{q} \wedge \mathrm{d}\boldsymbol{p}$ is preserved along the flow. Equivalently, the Lagrangian $L(\boldsymbol{q}, \dot{\boldsymbol{q}}, t) = T - V$ satisfies the Euler–Lagrange equations

$$\frac{\mathrm{d}}{\mathrm{d}t}\frac{\partial L}{\partial \dot{\boldsymbol{q}}} - \frac{\partial L}{\partial \boldsymbol{q}} = 0. \tag{1.5}$$

Noether's theorem connects continuous symmetries to conserved quantities — a principle that motivates both equivariant neural architectures (Chapter 8) and Hamiltonian/Lagrangian neural networks (Chapter 3).

## 1.5 Partial differential equations

A PDE is a relation $\mathcal{N}[u](\boldsymbol{x}, t) = 0$ on a domain $\Omega \subset \mathbb{R}^d$, with boundary conditions on $\partial\Omega$ and (for evolution problems) initial conditions at $t = 0$. Major classes:

- **Elliptic:** $-\Delta u = f$ (Poisson), steady-state.
- **Parabolic:** $\partial_t u = \Delta u + f$ (heat), diffusive.
- **Hyperbolic:** $\partial_t^2 u = c^2 \Delta u$ (wave), finite propagation speed.
- **Conservation laws:** $\partial_t u + \nabla \cdot \boldsymbol{F}(u) = 0$ (Burgers, Euler, Navier–Stokes).

Well-posedness in the sense of Hadamard requires existence, uniqueness, and continuous dependence on data. For nonlinear hyperbolic laws, *weak* (entropy) solutions handle shock formation; we will return to this when discussing PINN failure modes (Section 4.4).

## 1.6 Function spaces and the operator picture

Solutions of PDEs live in function spaces (Sobolev, Bochner, etc.). The map from problem data (coefficients, forcing, BCs, ICs) to solution

$$\mathcal{G} : \mathcal{X} \to \mathcal{Y}, \qquad a \mapsto u \tag{1.6}$$

is the *solution operator*. Most modern SciML — especially neural operators (Chapter 5) — aims to learn $\mathcal{G}$ directly, rather than its values $u$ for fixed $a$.

**Recipe: Picking the right viewpoint**

1. Single trajectory, fixed parameters: ODE/PDE *solver*.
2. Family of trajectories, varying ICs: *flow map* learning (neural ODE, time-stepping operator).
3. Family of problem instances: *solution operator* learning (DeepONet, FNO).
4. Sparse data, unknown equation: *system identification* (SINDy, UDEs).

## 1.7 Stochastic dynamics

When noise matters, replace the ODE by an SDE

$$\mathrm{d}\boldsymbol{X}_t = \boldsymbol{f}(\boldsymbol{X}_t, t)\mathrm{d}t + \boldsymbol{\sigma}(\boldsymbol{X}_t, t)\mathrm{d}\boldsymbol{W}_t, \tag{1.7}$$

with $\boldsymbol{W}_t$ a Wiener process. The Fokker–Planck equation governs the density:

$$\partial_t p = -\nabla\cdot(\boldsymbol{f}p) + \tfrac{1}{2}\nabla\nabla : (\boldsymbol{\sigma}\boldsymbol{\sigma}^{\mathsf{T}}p). \tag{1.8}$$

This connection between trajectories and densities is central to *score-based diffusion models* (Chapter 9), where the reverse-time SDE is parameterised by a learned score $\nabla \log p_t$.

## 1.8 Inverse problems

Many SciML applications are inverse: given observations $\boldsymbol{y} = \mathcal{H}\boldsymbol{u} + \boldsymbol{\eta}$ of a state governed by a PDE, infer parameters or initial conditions. Tikhonov regularisation, Bayesian inference, and adjoint methods provide the classical scaffolding; deep learning provides priors (deep image prior, score priors) and amortised solvers.

## 1.9 Summary

Dynamical systems theory provides the language of states, flows, stability, and conservation. Numerical analysis (Chapter 2) provides tools to compute. Modern SciML threads neural networks through both: as solvers, as surrogates, as identifiers, and as priors.

# 2 Classical Numerical Methods

*A good approximation is one that holds together for a while, and then breaks in a way you understand.*

Folklore in numerical analysis

A SciML practitioner cannot escape numerical analysis: it furnishes the training data, the baselines, and the inductive biases for many neural-network methods.

## 2.1 Time integration of ODEs

Given $\dot{\boldsymbol{x}} = \boldsymbol{f}(\boldsymbol{x}, t)$, denote the time step $h$.

### 2.1.1 Explicit Runge–Kutta

The classical $s$-stage RK scheme is

$$\boldsymbol{k}_i = \boldsymbol{f}\left(\boldsymbol{x}_n + h\sum_{j=1}^{i-1} a_{ij}\boldsymbol{k}_j,\ t_n + c_i h\right), \quad \boldsymbol{x}_{n+1} = \boldsymbol{x}_n + h\sum_{i=1}^{s} b_i \boldsymbol{k}_i. \tag{2.1}$$

RK4 ($s = 4$) achieves order four with the well-known Butcher tableau. Adaptive Dormand–Prince (DP5) embeds an order-4 estimator inside an order-5 step for error control.

### 2.1.2 Implicit and stiff solvers

Stiff systems — where the spectrum of the Jacobian spans many orders of magnitude — demand implicit methods. Backward differentiation formulas (BDF), implicit RK (Radau, SDIRK), and Rosenbrock methods are standard. The cost is a (possibly nonlinear) solve at each step.

**Insight: Why "stiff" bites neural ODEs**

Neural ODEs use *black-box* adaptive solvers and back-propagate through them. When the learned dynamics become stiff, solver step counts (and wall time) explode. Mitigations include semi-implicit adjoints, regularising stiffness, or restricting parameterisations (Chapter 3).

### 2.1.3 Symplectic integrators

For Hamiltonian systems, preserving $\omega$ matters more than local order. *Störmer–Verlet* and higher-order *Yoshida* splittings have bounded energy error over exponentially long times – critical for n-body astrophysics.

**Recipe: Choosing a time integrator**

1. Non-stiff, non-conservative: RK4, DP5, Tsit5.
2. Stiff: BDF, Radau IIA, Rosenbrock–Wanner.
3. Hamiltonian / long-horizon energy fidelity: Verlet, Yoshida-4.
4. Stochastic: Euler–Maruyama (order $\frac{1}{2}$), Milstein (order 1), stochastic Runge–Kutta.

## 2.2 Spatial discretisation of PDEs

### 2.2.1 Finite differences (FD)

Replace derivatives by stencils on a uniform grid: $\partial_x u_i \approx (u_{i+1} - u_{i-1})/(2h)$. Easy to implement, limited geometric flexibility, suffer at sharp gradients without upwinding or limiters.

### 2.2.2 Finite volumes (FV)

Integrate conservation laws over control volumes; fluxes at faces are reconstructed via Godunov / Roe / HLLC solvers. Native handling of shocks and discrete conservation. Backbone of CFD codes.

### 2.2.3 Finite elements (FE)

Project the weak form $\int_\Omega (\mathcal{L}u)v \,\mathrm{d}\Omega = \int_\Omega f v \,\mathrm{d}\Omega$ onto a finite-dimensional basis $\{\phi_j\}$. Galerkin / Petrov–Galerkin / DG variants. Excellent on complex geometries.

### 2.2.4 Spectral methods

Expand $u(\boldsymbol{x}) = \sum_k \hat{u}_k \phi_k(\boldsymbol{x})$ in a global basis (Fourier on tori, Chebyshev on intervals). Exponential convergence for analytic solutions; foundational for Fourier neural operators (Section 5.3).

### 2.2.5 Mesh-free / particle methods

Smoothed particle hydrodynamics (SPH), radial basis functions (RBF), and reproducing kernel particle methods. Neural networks themselves can be viewed as a mesh-free basis.

## 2.3 Linear algebra and Krylov methods

Most PDE discretisations reduce to large sparse linear systems $\boldsymbol{A}\boldsymbol{u} = \boldsymbol{b}$. Conjugate gradient (CG), GMRES, BiCGStab, and multigrid are workhorses. Preconditioning ($\boldsymbol{M}^{-1}\boldsymbol{A}\boldsymbol{u} = \boldsymbol{M}^{-1}\boldsymbol{b}$) is often the difference between a usable and unusable method.

## 2.4 Adjoint and sensitivity analysis

For gradient-based learning of physical parameters, the *continuous adjoint* computes $\mathrm{d}L/\mathrm{d}\mu$ at cost independent of $\dim \mu$ via a backward ODE:

$$\dot{\boldsymbol{\lambda}} = -\boldsymbol{\lambda}^\top \frac{\partial \boldsymbol{f}}{\partial \boldsymbol{x}} - \frac{\partial \ell}{\partial \boldsymbol{x}}, \qquad \frac{\mathrm{d}L}{\mathrm{d}\mu} = \int_0^T \boldsymbol{\lambda}^\top \frac{\partial \boldsymbol{f}}{\partial \mu}\,\mathrm{d}t. \tag{2.2}$$

The discrete adjoint differentiates the solver itself; both are special cases of automatic differentiation (Appendix A).

## 2.5 Stability, consistency, convergence

The Lax equivalence theorem ties these together for linear problems: *a consistent finite-difference scheme is convergent if and only if it is stable.* The CFL condition $|c|h_t \leq h_x$ for advection, the stiffness-driven step restriction for explicit parabolic schemes, and the von Neumann amplification analysis are all canonical. These constraints transfer almost verbatim to neural-network surrogates that mimic explicit time-stepping.

**Pitfall: Surrogate "solvers" that violate CFL**

A learned time-stepper trained on $\Delta t$ slightly larger than the finest physical time scale may appear accurate in-distribution but diverge catastrophically when rolled out — exactly the failure mode of an unstable explicit scheme. *Train-time rollouts* or implicit-style architectures help.

## 2.6 Reduced-order models (ROMs)

For high-dimensional systems, project onto a low-dimensional subspace. Proper Orthogonal Decomposition (POD) builds modes from snapshots via SVD. Galerkin or Petrov–Galerkin projection yields a small ODE on the modes. ROMs are often the bridge between full-order numerics and neural-operator surrogates — and provide Koopman/DMD with their data (Chapter 6).

## 2.7 Summary

Classical numerics gives us a vocabulary — consistency, stability, order, conservation, symplecticity — that survives intact in modern SciML, even when the basis functions are replaced by neural networks. The interesting question is not whether to abandon classical methods, but *which of their structural properties to preserve* inside a learned model.

# 3 Neural ODEs, CDEs, and SDEs

*A residual block is a discrete Euler step. Take the limit and you have an ODE.*

Chen, Rubanova, Bettencourt, Duvenaud (2018)

## 3.1 From ResNets to Neural ODEs

A residual block computes $\boldsymbol{h}_{n+1} = \boldsymbol{h}_n + \boldsymbol{f}_\theta(\boldsymbol{h}_n)$. Treating depth as time and taking $\Delta t \to 0$ gives the *neural ODE* [1]:

$$\frac{\mathrm{d}\boldsymbol{h}(t)}{\mathrm{d}t} = \boldsymbol{f}_\theta(\boldsymbol{h}(t), t), \qquad \boldsymbol{h}(0) = \boldsymbol{x}. \tag{3.1}$$

The "forward pass" is an ODE solve from 0 to $T$. Memory cost is $\mathcal{O}(1)$ in the number of steps because gradients are computed by solving a backward adjoint ODE rather than storing intermediate states.

### 3.1.1 Adjoint method

Define the loss $L(\boldsymbol{h}(T))$. Introduce the adjoint $\boldsymbol{a}(t) = \partial L / \partial \boldsymbol{h}(t)$, which satisfies

$$\frac{\mathrm{d}\boldsymbol{a}(t)}{\mathrm{d}t} = -\boldsymbol{a}(t)^\mathsf{T} \frac{\partial \boldsymbol{f}_\theta}{\partial \boldsymbol{h}}. \tag{3.2}$$

The parameter gradient is $\mathrm{d}L/\mathrm{d}\theta = -\int_0^T \boldsymbol{a}(t)^\mathsf{T} (\partial \boldsymbol{f}_\theta / \partial \theta)\, \mathrm{d}t$. This is the continuous-time analogue of backpropagation.

**Pitfall: Adjoint instability**

The continuous adjoint can be numerically inaccurate when the forward solver uses adaptive step control or interpolated dense output. The *interpolated* or *checkpointed* adjoint, and JAX/PyTorch `torchdiffeq` backsolve, mitigate this. For chaotic systems, adjoints diverge along Lyapunov directions; consider shadowing / ensemble adjoints.

## 3.2 Variants and architectures

**Augmented neural ODEs** The flow of an ODE is a homeomorphism, so a neural ODE cannot represent maps that change topology. Augmenting the state with extra dimensions $\boldsymbol{a}(0) = \boldsymbol{0}$ restores expressivity [2].

**Latent ODEs and continuous normalising flows.** For irregular time series and density estimation, model dynamics in a latent space and decode [3]. Continuous normalising flows (CNFs) use the instantaneous change-of-variables formula $\mathrm{d}\log p/\mathrm{d}t = -\operatorname{Tr}(\partial \boldsymbol{f}/\partial \boldsymbol{h})$ to evaluate likelihoods.

**Neural CDEs.** For irregular time series, treat the data path $\boldsymbol{X}(t)$ as a control and integrate $\mathrm{d}\boldsymbol{h}(t) = \boldsymbol{f}_\theta(\boldsymbol{h}(t))\,\mathrm{d}\boldsymbol{X}(t)$ in the sense of controlled differential equations [4].

**Neural SDEs.** Add Brownian motion: $\mathrm{d}\boldsymbol{h} = \boldsymbol{f}_\theta\,\mathrm{d}t + \boldsymbol{\sigma}_\theta\,\mathrm{d}\boldsymbol{W}_t$. Trainable via Wasserstein matching, KL with reference SDE, or score-matching; foundational for generative diffusion (Chapter 9).

## 3.3 Hamiltonian and Lagrangian neural networks

Architectural priors that respect mechanics:

- **HNN** [5]: parameterise $H_\theta(\boldsymbol{q}, \boldsymbol{p})$; integrate $\dot{\boldsymbol{q}} = \partial_{\boldsymbol{p}} H_\theta$, $\dot{\boldsymbol{p}} = -\partial_{\boldsymbol{q}} H_\theta$.
- **LNN** [6]: parameterise $L_\theta(\boldsymbol{q}, \dot{\boldsymbol{q}})$ and impose Euler–Lagrange equations.
- **Symplectic ODE-Net, Constrained HNN**: enforce constraints and conservation explicitly.

These networks generalise far beyond their training horizon precisely because they encode the right invariant.

## 3.4 ODE-based deep generative models

### 3.4.1 Continuous normalising flows (CNF)

Train $\boldsymbol{f}_\theta$ so that the pushforward of a base density matches the data density:

$$\log p_\theta(\boldsymbol{x}) = \log p_0(\boldsymbol{h}(0)) - \int_0^T \nabla\cdot \boldsymbol{f}_\theta(\boldsymbol{h}(t), t)\,\mathrm{d}t. \tag{3.3}$$

### 3.4.2 FFJORD and Hutchinson trace estimators

Replace the exact trace by a Hutchinson estimator $\operatorname{Tr}(\boldsymbol{J}) \approx \mathbb{E}_{\boldsymbol{\epsilon}\sim\mathcal{N}}[\boldsymbol{\epsilon}^\top \boldsymbol{J}\boldsymbol{\epsilon}]$ for tractable training.

### 3.4.3 Flow matching and stochastic interpolants

Recent advances [7, 8] train CNFs by regressing onto a target velocity field defined by a prescribed interpolation between base and data distributions, avoiding expensive ODE simulation during training. Discussed at length in Chapter 9.

## 3.5 Practical training tips

**Recipe: Training a neural ODE end-to-end**

1. Normalise inputs/outputs; centre time near zero.
2. Start with a permissive solver (Tsit5, atol/rtol $10^{-3}$) and tighten as training progresses.
3. Use the *checkpointed adjoint* for long horizons; the *recursive checkpoint* for memory.
4. Add Jacobian/Frobenius penalties to discourage stiffness.

5. Curriculum on $T$: train on short windows, extend gradually.

## 3.6 Where neural ODEs shine and where they struggle

**Strengths:** natural for irregular time series, continuous-depth generative modelling, mechanistic priors via HNN/LNN, smooth and invertible models.
**Weaknesses:** high wall-clock cost from solver calls; representation limited to homeomorphisms (without augmentation); delicate adjoint behaviour on stiff or chaotic dynamics.

**Insight: Neural ODE = parametric differential equation**

The most productive way to think about neural ODEs is not as a deep network whose depth is continuous, but as a *parametric ODE family fitted to data*. This perspective unifies them with Universal Differential Equations (Chapter 7), system identification (Chapter 6), and operator learning (Chapter 5).

# 4

# Physics-Informed Neural Networks

*A PINN is a neural network whose loss function is a PDE.*

Raissi, Perdikaris, Karniadakis (2019)

## 4.1 Formulation

Consider a PDE on $\Omega \times [0, T]$:

$$\mathcal{N}[u](\boldsymbol{x}, t) = 0 \quad \text{in } \Omega \times (0, T], \tag{4.1}$$

$$\mathcal{B}[u](\boldsymbol{x}, t) = g(\boldsymbol{x}, t) \quad \text{on } \partial\Omega \times [0, T], \tag{4.2}$$

$$u(\boldsymbol{x}, 0) = u_0(\boldsymbol{x}) \quad \text{in } \Omega. \tag{4.3}$$

A PINN parameterises $u \approx u_\theta : \mathbb{R}^{d+1} \to \mathbb{R}^k$ as a multilayer perceptron and minimises a composite loss [9]

$$\mathcal{L}(\theta) = w_r \mathcal{L}_r + w_b \mathcal{L}_b + w_0 \mathcal{L}_0 + w_d \mathcal{L}_d, \tag{4.4}$$

where

$$\mathcal{L}_r = \frac{1}{N_r} \sum_i \left| \mathcal{N}[u_\theta](\boldsymbol{x}_i, t_i) \right|^2, \tag{4.5}$$

$$\mathcal{L}_b = \frac{1}{N_b} \sum_i \left| \mathcal{B}[u_\theta] - g \right|^2, \tag{4.6}$$

$$\mathcal{L}_0 = \frac{1}{N_0} \sum_i \left| u_\theta(\boldsymbol{x}_i, 0) - u_0 \right|^2, \tag{4.7}$$

$$\mathcal{L}_d = \frac{1}{N_d} \sum_i \left| u_\theta(\boldsymbol{x}_i, t_i) - u_i^{\text{obs}} \right|^2. \tag{4.8}$$

Derivatives appearing in $\mathcal{N}$ are obtained by automatic differentiation through the network.

## 4.2 Why it works (when it works)

The PINN loss imposes the strong-form PDE on a finite set of *collocation points*. With sufficiently expressive $u_\theta$ and adequate optimisation, $u_\theta$ approaches a strong solution. The formulation is mesh-free, accommodates irregular geometries, and unifies forward and inverse problems — inverse coefficients become trainable parameters.

## 4.3 Variants

**XPINN / cPINN.** Domain decomposition: separate networks per subdomain stitched together by interface losses, enabling parallelism and capture of localised features [10].

**hp-VPINN.** Use the variational (weak) form with test functions, integrating by parts to reduce derivative orders — helpful when solutions have low regularity.

**Causal PINN.** Re-weight the time-domain residual to enforce that the loss at later times is small only *after* earlier times converge, avoiding non-causal solutions to time-evolution PDEs [11].

**NTK-based PINN.** The neural tangent kernel reveals spectral bias in PINN training: high-frequency residual modes converge slowly. Re-weighting based on the NTK eigenvalues fixes the imbalance [12].

**Self-adaptive / SA-PINN.** Per-point trainable weights $\lambda_i$ adversarially upweight hard collocation points [13].

**PINN-LS, gPINN, PIELM.** Least-squares formulations, gradient-enhanced PINNs, and physics-informed extreme learning machines trade off flexibility for speed.

## 4.4 Failure modes

PINNs are notoriously brittle. Documented failure modes include:

1. **Spectral bias** towards low frequencies; sharp gradients and shock fronts under-resolved.
2. **Multi-objective imbalance**: residual, boundary, and data losses conflict; gradient pathologies emerge.
3. **Stiffness in time**: optimiser converges to spurious steady states matching the residual at $t = 0$ but not propagating.
4. **Non-convex landscapes**: local minima where residual is small but solution is wrong.
5. **Floating-point precision**: a 2025 result shows many "failure modes" disappear when training is performed in `float64` rather than `float32`, because the L-BFGS convergence test is otherwise triggered prematurely [14].

**Pitfall: The "trivial solution" trap**

For homogeneous PDEs with zero forcing and inadequate boundary loss weighting, a PINN can converge to $u_\theta \equiv 0$. Always sanity-check the solution magnitude and inspect residual histograms by region.

### 4.4.1 Mitigations

**Recipe: Hardening a PINN**

1. Use `float64`; verify L-BFGS convergence flags.
2. Adaptive collocation: resample where residual is large (RAR, RAD).
3. Loss balancing: NTK weights, gradient-norm balancing, SoftAdapt, ReLoBRaLo.
4. Hard-constrain BCs/ICs by output transforms $u_\theta(\boldsymbol{x}, t) = u_0(\boldsymbol{x}) + t\,\boldsymbol{N}_\theta$.

5. Causal time-marching: train on expanding time windows.
6. Fourier features / SIREN activations to combat spectral bias.
7. Exact gradient computation; avoid `create_graph=True` pitfalls in PyTorch.

## 4.5 Inverse problems with PINNs

PINNs shine in data-assimilation regimes where partial measurements constrain unknown coefficients $\mu$ in $\mathcal{N}_\mu$. Joint optimisation of $(\theta, \mu)$ gives forward + inverse for free. This is why PINNs are popular in subsurface flow, biomechanics, and astrophysical fluid inference.

## 4.6 Theoretical guarantees

Universal approximation extends to PINNs under mild regularity, but *trainability* is the binding constraint. Recent work bounds the generalisation error in terms of the residual loss, neural tangent kernel conditioning, and quadrature error of the collocation sampling [12, 15].

## 4.7 When to choose a PINN over alternatives

**Insight: PINNs vs. neural operators vs. classical solvers**

- Single problem instance, smooth solution, sparse data, want mesh-free + inverse parameters: *PINN*.
- Many problem instances, repeated queries: *neural operator* (Chapter 5).
- One-shot accuracy, complex geometry, no parameter unknowns: classical FE/FV solver.
- Stiff multiscale + partial data: *universal differential equations* (Chapter 7).

## 4.8 Code skeleton (JAX)

```python
import jax, jax.numpy as jnp, optax
from flax import linen as nn

class MLP(nn.Module):
    feats: tuple = (64,)*5
    @nn.compact
    def __call__(self, x):
        for f in self.feats:
            x = nn.tanh(nn.Dense(f)(x))
        return nn.Dense(1)(x)

def u_fn(params, x, t):
    return model.apply(params, jnp.stack([x, t]))[0]

def residual(params, x, t, nu=0.01/jnp.pi):
    u   = u_fn(params, x, t)
    u_t = jax.grad(u_fn, argnums=2)(params, x, t)
    u_x = jax.grad(u_fn, argnums=1)(params, x, t)
    u_xx= jax.grad(lambda p,x,t:
        jax.grad(u_fn,argnums=1)(p,x,t),argnums=1)(params,x,t)
    return u_t + u*u_x - nu*u_xx
```

Listing 4.1: Minimal PINN for 1D Burgers

## 4.9 Summary

PINNs remain a swiss-army knife when the equation is known, geometry is awkward, and data is scarce. Their reputation for fragility is partly fair and partly due to under-engineered training. With float64, adaptive sampling, balanced losses, and causal training, vanilla PINNs solve a remarkable range of problems — a fact that the literature has recently re-affirmed [14].

# 5 Neural Operators

*Operator learning is to function approximation what calculus is to arithmetic.*

Anandkumar, Stuart and collaborators

## 5.1 Solution operators

The forward map of a parameterised PDE family is an operator

$$\mathcal{G} : a \in \mathcal{A} \longmapsto u \in \mathcal{U}, \tag{5.1}$$

between Banach spaces of functions. Where a PINN learns one $u$, a *neural operator* learns $\mathcal{G}$. Once trained, evaluating $\mathcal{G}$ on a new $a$ is a forward pass — orders of magnitude faster than a fresh PDE solve.

## 5.2 DeepONet

DeepONet [16] factorises the operator into branch and trunk networks:

$$\mathcal{G}[a](\boldsymbol{y}) \approx \sum_{k=1}^{p} b_k(a(\boldsymbol{x}_1), \ldots, a(\boldsymbol{x}_m))\, t_k(\boldsymbol{y}), \tag{5.2}$$

where the *branch* $\boldsymbol{b}$ encodes the input function (sampled at sensors $\boldsymbol{x}_i$) and the *trunk* $\boldsymbol{t}$ encodes the query point $\boldsymbol{y}$. The architecture is justified by a universal approximation theorem for nonlinear continuous operators (Chen & Chen 1995). Variants: POD-DeepONet, multiple-input DeepONet, physics-informed DeepONet.

## 5.3 Fourier Neural Operator (FNO)

FNO [17] parameterises the operator as a sequence of layers

$$v_{\ell+1}(\boldsymbol{x}) = \sigma\Big(\boldsymbol{W} v_\ell(\boldsymbol{x}) + \mathcal{F}^{-1}(\boldsymbol{R}_\ell \cdot \mathcal{F} v_\ell)(\boldsymbol{x})\Big)\,, \tag{5.3}$$

where $\mathcal{F}$ is the Fourier transform and $\boldsymbol{R}_\ell$ is a learned truncated convolution kernel acting on the lowest $K$ modes. Key properties:

- Discretisation invariance: trained on one resolution, queried at another (with caveats).
- Quasi-linear cost via FFT: $\mathcal{O}(N \log N)$ per layer.
- Strong empirical performance on smooth periodic problems.

### 5.3.1 Variants of FNO

**Geo-FNO, Factorised FNO, Tensor-FNO.** Handle non-rectangular domains by warping geometries; factorise modes across dimensions; tensor-decompose the kernel.

**U-FNO, U-Net FNO.** Combine FNO with U-Net hierarchy for multiscale features.

**Spherical FNO** [18]. Replace FFT with spherical harmonic transforms; used for state-of-the-art global weather emulators (FourCastNet, GraphCast hybrids).

**Adaptive FNO and AFNO.** Adapt mode truncation per token, used inside vision transformers.

**Pitfall: FNO discretisation invariance, with asterisks**

Out-of-distribution behaviour at resolutions far from training is dataset-dependent. Aliasing, periodic-padding artefacts, and limited mode budgets can all break the "train once, query anywhere" promise. Always benchmark on multiple resolutions.

## 5.4 Graph Neural Operators

For unstructured meshes, GNOs [19, 20] parameterise an integral operator

$$v_{\ell+1}(\boldsymbol{x}) = \sigma\left(\boldsymbol{W} v_\ell(\boldsymbol{x}) + \int_{B_r(\boldsymbol{x})} \kappa_\theta(\boldsymbol{x}, \boldsymbol{y}) v_\ell(\boldsymbol{y})\, \mathrm{d}\boldsymbol{y}\right) \tag{5.4}$$

via message passing on a graph built from the mesh. MeshGraphNet, GNN-FNO hybrids, and the message-passing PDE solver belong to this family.

## 5.5 Transformer-based operators

Transformers naturally handle variable-length, irregular inputs. Recent high-impact systems include:

- **GNOT, OFormer, GAOT**: cross-attention between query points and conditioning.
- **Transolver**: token-mixing in physical space with attention across slices.
- **DPOT, MPP, Poseidon**: transformer-based foundation models (Chapter 13).

**Poseidon** [21] is a multiscale operator transformer pretrained on a heterogeneous mix of PDE families with *time-conditioned layer norms* and a semi-group training loss; it demonstrates strong few-shot transfer to new PDEs.

## 5.6 Discretisation invariance: theory

A defining requirement of a *neural operator* (vs. a discretisation-tied surrogate) is that it is well-defined as a map between function spaces, with finite-dimensional implementations recovered as projections [22]. Universal approximation theorems exist for FNO, DeepONet, and graph operators under appropriate compactness assumptions.

## 5.7 Training data

**Recipe: Building a neural-operator dataset**

1. Define the PDE family by sampling parameters/coefficients/ICs from a distribution that covers downstream use.
2. Solve with a high-fidelity classical solver; store inputs $a_i$ and solutions $u_i$ at multiple resolutions / time steps.
3. Decide loss: relative $L^2$, $H^1$, or task-specific energy norms.
4. For time-dependent operators, choose autoregressive (next-step) or direct (input-to-final) framing; combine with rollout regularisation.
5. Augment with symmetries (translation, rotation, reflection) when the PDE allows.

## 5.8 Beyond pure data: physics-informed operators

Add residual losses on the operator output:

$$\mathcal{L}_{\text{op}} = \mathbb{E}_a \left[ \|\mathcal{G}_\theta[a] - u_a\|^2 + \lambda \|\mathcal{N}_a[\mathcal{G}_\theta[a]]\|^2 \right]. \tag{5.5}$$

PI-DeepONet, PINO, and FNO-PINN hybrids report improved generalisation and faster convergence with limited data.

## 5.9 Limitations and open problems

- Generalisation to unseen geometries / boundary conditions.
- High-frequency / shock fronts: most operators are biased to smooth solutions.
- Long-horizon temporal stability: errors accumulate in autoregressive rollout.
- Foundation-model scaling: data heterogeneity is harder than in language; tokenisation of physical fields is unsolved.

## 5.10 Choosing an operator architecture

**Insight: Quick decision guide**

- Periodic / regular grid, smooth solutions: *FNO*.
- Sphere / earth-system: *SFNO*.
- Branch–trunk separation, point queries, branched inputs: *DeepONet*.
- Unstructured mesh / complex geometry: *GNO / MGN / Transolver*.
- Few-shot transfer across PDE families: *Poseidon, DPOT, MPP* (foundation operators).

# 6

# Operator-Theoretic Learning: Koopman, DMD, SINDy

*Linearity is a virtue. Even nonlinear systems are linear — in infinite dimensions.*

Bernard Koopman, 1931

## 6.1 The Koopman operator

For a discrete dynamical system $\boldsymbol{x}_{k+1} = \boldsymbol{F}(\boldsymbol{x}_k)$ on a state space $M$, the *Koopman operator* $\mathcal{K}$ acts on observables $g : M \to \mathbb{C}$:

$$(\mathcal{K}g)(\boldsymbol{x}) = g(\boldsymbol{F}(\boldsymbol{x})). \tag{6.1}$$

$\mathcal{K}$ is linear (even when $\boldsymbol{F}$ is nonlinear) but infinite-dimensional. Its spectrum encodes invariant sets, modes, and ergodic properties.

### 6.1.1 Dynamic Mode Decomposition (DMD)

Given snapshots $\boldsymbol{X} = [\boldsymbol{x}_0, \ldots, \boldsymbol{x}_{m-1}]$, $\boldsymbol{Y} = [\boldsymbol{x}_1, \ldots, \boldsymbol{x}_m]$, DMD seeks $\boldsymbol{A}$ minimising $\|\boldsymbol{Y} - \boldsymbol{A}\boldsymbol{X}\|_F$, giving $\boldsymbol{A} = \boldsymbol{Y}\boldsymbol{X}^\dagger$. The eigendecomposition of $\boldsymbol{A}$ yields DMD modes (spatial structures) and DMD eigenvalues (frequencies and growth rates).

### 6.1.2 Extended DMD (eDMD)

Lift state via a dictionary $\boldsymbol{\psi}(\boldsymbol{x}) = (\psi_1(\boldsymbol{x}), \ldots, \psi_N(\boldsymbol{x}))$ and regress $\boldsymbol{\psi}(\boldsymbol{x}_{k+1}) \approx \boldsymbol{K}\boldsymbol{\psi}(\boldsymbol{x}_k)$. For polynomial dictionaries this recovers SINDy; for radial bases / random features this recovers kernel DMD.

### 6.1.3 Deep Koopman

Replace the dictionary by a learned encoder $\boldsymbol{\psi}_\theta : M \to \mathbb{C}^N$ with a *linear* latent dynamics $\boldsymbol{z}_{k+1} = \boldsymbol{K}_\theta \boldsymbol{z}_k$ and a decoder $\boldsymbol{\phi}_\theta$. Loss combines reconstruction, prediction, and linearity terms [23].

**Insight: Why Koopman is powerful for ML**

- Eigenfunctions provide globally meaningful coordinates – "intrinsic" phase variables.
- Linearity in latent space yields easy long-horizon prediction, control, and stability analysis.

- Captures multi-scale temporal structure via Koopman spectra.

But *spectral pollution* — spurious eigenvalues from finite truncation — is a persistent danger. Modern remedies include ResDMD, EDMD-DL with closure error, and residual-based residual estimators.

## 6.2 Sparse identification of nonlinear dynamics (SINDy)

SINDy [24] fits the ODE $\dot{\boldsymbol{x}} = \boldsymbol{f}(\boldsymbol{x})$ in a sparse linear combination of a candidate library $\boldsymbol{\Theta}(\boldsymbol{x})$:

$$\dot{\boldsymbol{X}} = \boldsymbol{\Theta}(\boldsymbol{X})\boldsymbol{\Xi}, \qquad \boldsymbol{\Xi} = \arg\min_{\boldsymbol{\Xi}} \|\dot{\boldsymbol{X}} - \boldsymbol{\Theta}(\boldsymbol{X})\boldsymbol{\Xi}\|_2^2 + \lambda\|\boldsymbol{\Xi}\|_0. \tag{6.2}$$

$\ell_0$ is approximated by sequential thresholding (STLS), LASSO, or sparse relaxed regularised regression (SR3).

### 6.2.1 Variants

**Weak-form / Integral SINDy.** Multiply the ODE by test functions and integrate by parts to avoid finite-difference noise on $\dot{\boldsymbol{x}}$. Weak-SINDy and PDE-FIND for PDEs.

**Constrained SINDy.** Enforce conservation, energy, or symmetry by linear constraints on $\boldsymbol{\Xi}$.

**Ensemble SINDy / E-SINDy.** Bootstrap the data and library to obtain robust models with uncertainty estimates.

**Implicit SINDy.** Identify rational models $f = \boldsymbol{\Theta}_n\boldsymbol{\xi}_n/\boldsymbol{\Theta}_d\boldsymbol{\xi}_d$.

**SINDy-AE.** Wrap SINDy in an autoencoder to discover dynamics in a learned coordinate system.

### 6.2.2 Pitfalls

**Pitfall: Library construction is the whole game**

SINDy is interpretable only if the true dynamics live in your library. Choosing too small a library forces wrong models; too large makes sparsity selection brittle. Domain knowledge wins.

## 6.3 Symbolic regression

Where SINDy fixes a candidate library, symbolic regression searches the space of expression trees (operators, variables, constants):

- **Genetic programming**: PySR, Eureqa.
- **AI Feynman**: combines neural fits, dimensional analysis, and combinatorial search to recover Feynman lectures' equations.
- **Transformer-based symbolic regression**: SymbolicGPT, NeSymRes, end-to-end equation generation.
- **KAN-SR**: Kolmogorov–Arnold networks (Chapter 7) yield analytic primitives via spline activations.

Symbolic regression provides interpretable physics — closing the loop from data to equations.

## 6.4 Reservoir computing and echo state networks

Random nonlinear recurrent reservoirs followed by a linear readout fit chaotic dynamics surprisingly well, with $\mathcal{O}$(seconds) training time. Particularly good for short-term forecasting of high-dimensional spatiotemporal chaos (Lorenz-96, Kuramoto–Sivashinsky), and as a baseline against deep models.

## 6.5 Recipe: pick the right operator-theoretic tool

**Recipe: Decision flow**

1. Have snapshots of a single trajectory and want modes? *DMD*.
2. Want a global linear model on lifted observables? *eDMD / deep Koopman*.
3. Suspect the dynamics are sparse polynomials in known variables? *SINDy*.
4. No idea of the right form, want a closed-form expression? *symbolic regression*.
5. Just want a fast forecaster on a chaotic system? *reservoir computing* as baseline.

## 6.6 Connections to neural operators

Koopman is, abstractly, an *operator on observables*; FNO and DeepONet are *operators on PDE solution states*. Both reformulate nonlinear dynamics as linear/operator problems; the difference is the ambient space (point observables vs. function fields). Several works connect the two perspectives, e.g. Koopman neural operators and spectral DMD-FNO hybrids.

# 7 Hybrid Models and Universal Differential Equations

*The differential equation is the model. The neural network is a closure.*

Rackauckas et al., SciML.ai

## 7.1 Universal differential equations

A *universal differential equation* (UDE) augments a known model with a neural surrogate for the unknown parts [25]:

$$\dot{\boldsymbol{x}} = \boldsymbol{f}(\boldsymbol{x}, t; \mu) + \boldsymbol{N}_\theta(\boldsymbol{x}, t), \tag{7.1}$$

where $\boldsymbol{f}$ encodes physics we believe (kinematics, conservation), and $\boldsymbol{N}_\theta$ absorbs missing dynamics — closure terms, unmodelled couplings, anomalous behaviour. Training is end-to-end via adjoint sensitivity through ODE solvers (Chapters 2 and 3).

### 7.1.1 Why hybrid beats both extremes

- Pure ODEs over-fit assumed structure and ignore observed discrepancies.
- Pure neural networks waste capacity rediscovering known physics and generalise poorly out of distribution.
- UDEs combine: low-data efficiency from physics, flexibility from the network. The learned residual is often *symbolic-regressible* into a closed-form correction.

### 7.1.2 Discovery loop

**Recipe: The Rackauckas–Edelman recipe**

1. Write down the known structural ODE/PDE.
2. Add $\boldsymbol{N}_\theta$ to the residual or specific terms.
3. Train end-to-end with adjoint gradients on observed data.
4. Apply SINDy / symbolic regression to $\boldsymbol{N}_\theta$ to extract analytic closure.
5. Substitute back, re-train, validate.

## 7.2 Closure modelling

Turbulence, plasma, and climate codes often suffer from unresolved small scales whose effects must be parameterised. Neural closure models replace classical Reynolds/Boussinesq closures with neural networks:

- **LES–ML closures**: subgrid-stress predicted by CNN/GNN.
- **Climate parameterisations**: convection, cloud, radiation schemes (e.g. NeuralGCM [26]).
- **Plasma turbulence**: gyrokinetic surrogates.

**NeuralGCM.** A 2024 hybrid global atmosphere model that couples a differentiable dynamical core with neural parameterisations and matches state-of-the-art weather forecast skill at a fraction of the compute [26]. A landmark for hybrid SciML at scale.

## 7.3 Differentiable simulators

A differentiable simulator exposes $\partial$output/$\partial$input through the entire physical pipeline (PDE solve, contact, collision, etc.). Combined with neural augmentation, this gives gradient-based parameter inference, control, and design optimisation:

- JAX-MD, JAX-CFD, JAX-Fluids, Phiflow, Modulus, Brax.
- DiffTaichi, Warp, Nimble, Drake (robotics).
- Diffrax + Equinox (Julia/JAX) for differentiable solvers.

## 7.4 Kolmogorov–Arnold Networks (KAN)

KANs [27] replace the affine $\boldsymbol{W}\boldsymbol{x} + \boldsymbol{b}$ in MLPs with learnable univariate spline activations on edges, motivated by the Kolmogorov–Arnold representation theorem. They claim:

- Improved interpretability via per-edge function visualisation.
- Sometimes superior accuracy on PDE / symbolic-regression benchmarks at small parameter counts.
- Direct symbolic extraction by fitting splines to elementary functions.

**Pitfall: KAN hype, KAN reality**

Independent benchmarks have shown that the gap between KANs and well-tuned MLPs is much smaller, and training cost is higher. Treat KANs as one more architectural option, not a panacea.

## 7.5 Operator + physics hybrids

- **PINO**: physics-informed neural operator combines FNO training data with PDE residuals.
- **PI-DeepONet**: residual loss for the trunk–branch decomposition.
- **Differentiable solver $\circ$ NN**: replace expensive solver substeps with neural surrogates inside an otherwise classical solver (multigrid smoothers, preconditioners, time-stepping).

## 7.6 Equation-discovery pipelines

Modern SciML pipelines string together:

$$\text{data} \to \text{denoising / smoothing} \to \text{candidate libraries} \to \text{sparse / symbolic fit} \to \text{validation}$$

with neural surrogates entering wherever the bottleneck lies. Auto-MoDe and AI-Hilbert (NeurIPS 2024) automate large parts of this loop.

## 7.7 Summary

Hybrid SciML is arguably the most pragmatic part of the field: mechanistic backbone + data-driven correction. It dominates applications where physics is partially known and computational budgets are real — climate, biology, plasma, materials.

# 8 Geometric and Equivariant Deep Learning

*Symmetry, as wide or as narrow as you may define its meaning, is one idea by which man through the ages has tried to comprehend and create order, beauty, and perfection.*

Hermann Weyl

## 8.1 Why symmetry matters

Physical systems exhibit symmetries: translations, rotations, reflections, Galilean boosts, gauge transformations. A model that respects these symmetries by construction (*equivariant*) requires less data, generalises better, and respects conservation laws — a direct consequence of Noether's theorem.

**Definition 8.1: Equivariance**

A map $\Phi : \mathcal{X} \to \mathcal{Y}$ is *equivariant* with respect to a group $G$ acting on $\mathcal{X}$ via $\rho_X$ and on $\mathcal{Y}$ via $\rho_Y$ if $\Phi(\rho_X(g)\boldsymbol{x}) = \rho_Y(g)\Phi(\boldsymbol{x})$ for all $g \in G$. Invariance is the special case $\rho_Y \equiv \mathrm{id}$.

## 8.2 Group-equivariant CNNs

G-CNNs generalise convolution: weights are functions on $G$ rather than $\mathbb{R}^d$, and convolution becomes integration over the group. Cohen & Welling (2016) introduced the framework for finite groups ($\mathbb{Z}_4$, dihedral). Steerable CNNs use representation theory to construct continuous-group equivariant filters.

## 8.3 $E(3)$ and $SE(3)$ equivariance

For 3D physics, $E(3)$ (translations + rotations + reflections) is the natural symmetry group. Two leading paradigms:

**Tensor field networks / SE(3)-Transformer / NequIP.** Activations are sums of irreducible representations $\mathbf{V} = \bigoplus_\ell \mathbf{V}^{(\ell)}$ of $SO(3)$. Layers combine these via tensor products and Clebsch–Gordan coefficients [28–30].

**E(n)-equivariant graph neural networks (EGNN).** Maintain scalar features $\boldsymbol{h}_i$ and equivariant coordinates $\boldsymbol{x}_i$; updates use only invariants like $\|\boldsymbol{x}_i - \boldsymbol{x}_j\|$. Lightweight and effective [31].

**Geometric algebra networks.** GATr (Brehmer et al.) operate in the projective geometric algebra of 3D Euclidean space, encoding rotations, translations, and reflections naturally.

## 8.4 N-body dynamics

Equivariant networks dominate learned $n$-body simulators:

- **Molecular dynamics**: NequIP, Allegro, MACE, SO3krates, GemNet — machine-learning force fields with $SO(3)$ equivariance.
- **Astrophysics**: equivariant graph networks for galaxy mergers, halo finders, cosmological parameter inference. Particularly relevant when conserving angular momentum and energy is critical for long-horizon stability.
- **Particle physics**: ParticleNet, LorentzNet (Lorentz equivariance) for jet tagging and event classification.

**Insight: Equivariance as inductive bias**

The single biggest lesson of geometric deep learning: *the right inductive bias outweighs more data.* NequIP/Allegro reach DFT accuracy on chemistry tasks with $\sim$10$\times$ less data than non-equivariant counterparts.

## 8.5 Hamiltonian / Lagrangian + equivariance

Combining mechanical priors with symmetries gives the strongest extrapolation. Examples:

- Constrained Hamiltonian neural networks for spherical pendulums.
- Lagrangian neural networks with $SE(3)$ equivariance for articulated robotics.
- Symplectic + equivariant ODE-Nets for n-body simulations.

## 8.6 Mesh-based and graph-based PDE surrogates

**MeshGraphNets [32]** Learn dynamics on mesh graphs with edge-conditioned message passing; flexible across geometries; foundational for many CFD, soft-body, and cloth simulators.

**Lagrangian particle GNNs [33].** Treat particles as nodes; predict accelerations; integrate explicitly. Used for fluids, granular media.

## 8.7 Open challenges

- Computational cost of high-$\ell$ tensor products (mitigated by MACE / SO3krates / Cartesian-tensor reformulations).
- Equivariance under *approximate* symmetries (e.g. slight gauge breaking).
- Combining symmetry with geometric heterogeneity (boundaries, defects).
- Foundation models that are simultaneously equivariant and large.

## 8.8 Recipe: choosing an equivariant model

**Recipe: Equivariant architecture selection**

1. Symmetry group of your problem (E(3)? SE(3)? Lorentz? gauge?).
2. Required output type (scalar invariant? vector? tensor?).
3. Cost budget: EGNN $<$ MACE $<$ TFN/SE(3)-T.
4. Conservation requirements: pair with HNN/LNN if energy/momentum conservation is crucial.

# 9 Generative Models for Dynamical Systems

*The future of simulation is sampling.*

(Folk wisdom, SciML circles, c. 2024)

## 9.1 Why generative?

Many physical systems are inherently stochastic (turbulence, multiphase flows, ensemble forecasts, climate). Even deterministic PDEs benefit from probabilistic surrogates that quantify uncertainty and recover multimodal distributions. Generative models provide *distributions over solutions* rather than point predictions.

## 9.2 Diffusion models

### 9.2.1 Score-based formulation

Define a forward SDE that perturbs data into noise:

$$\mathrm{d}\boldsymbol{X}_t = \boldsymbol{f}(\boldsymbol{X}_t, t)\mathrm{d}t + g(t)\mathrm{d}\boldsymbol{W}_t, \qquad t \in [0, T]. \tag{9.1}$$

The reverse SDE generating samples from $p_T$ back to $p_0$ is

$$\mathrm{d}\boldsymbol{X}_t = \left[\boldsymbol{f}(\boldsymbol{X}_t, t) - g(t)^2 \nabla \log p_t(\boldsymbol{X}_t)\right]\mathrm{d}t + g(t)\mathrm{d}\bar{\boldsymbol{W}}_t. \tag{9.2}$$

The score $\nabla \log p_t$ is learned by score matching [34]. Equivalent ODE formulation ("probability flow ODE") gives deterministic sampling.

### 9.2.2 Diffusion for PDEs

Diffusion models are increasingly used as PDE surrogates and super-resolvers:

- **PDE-Refiner** [35]: iterative noise denoising at multiple scales fixes spectral defects of single-shot operators.
- **DYffusion** [36]: temporal diffusion models for stochastic forecasting.
- **SimDiffPDE** [37]: image-to-image transformer diffusion as a unified PDE solver.
- **Diffusion priors with physics-guided inference** [38]: train a generic diffusion prior; condition at inference on PDE residual via guidance.
- **GenCast** [39]: diffusion-based ensemble weather forecasting outperforming top operational systems.

### 9.2.3 Conditional / guided diffusion

- **Classifier-free guidance**: train conditional and unconditional networks jointly.
- **Plug-and-play physics guidance**: at sampling, add a physics-based correction term $-\lambda\nabla\|\mathcal{N}[\hat{u}]\|^2$ to the score.
- **Posterior sampling for inverse problems** (DPS, ΠGDM): combine diffusion priors with likelihoods $p(\boldsymbol{y}|\boldsymbol{x})$ for data assimilation, super-resolution, sparse-view reconstruction.

## 9.3 Flow matching and stochastic interpolants

**Flow matching** [7]. Choose an interpolation $\boldsymbol{x}_t = \alpha_t \boldsymbol{x}_1 + \sigma_t \boldsymbol{x}_0$ between data $\boldsymbol{x}_1$ and noise $\boldsymbol{x}_0$, with target velocity $\boldsymbol{u}_t(\boldsymbol{x}) = \mathbb{E}[\dot{\boldsymbol{x}}_t|\boldsymbol{x}_t = \boldsymbol{x}]$. Train $\boldsymbol{v}_\theta(\boldsymbol{x}, t)$ by regression on conditional velocities — no SDE simulation, no score estimation.

**Stochastic interpolants** [8]. Generalise flow matching with arbitrary interpolation paths and noise schedules.

**Physics-Based Flow Matching (PBFM) [40].** Augment the flow-matching loss with PDE residuals and algebraic constraints, so the learned velocity field generates physically consistent samples.

## 9.4 Generative ensembles for weather and climate

A 2024–2025 milestone: ML-based ensemble systems (GenCast, NeuralGCM ensembles) match or surpass operational ensembles in skill while running orders of magnitude faster on accelerators. Implications for SciML:

- Foundation operators (Chapter 13) train deterministic surrogates; diffusion sits on top to provide ensemble spread.
- Calibration and reliability metrics (CRPS, rank histograms) become as important as RMSE.

## 9.5 GANs and energy-based models in SciML

While diffusion has overtaken GANs, mention:

- **Physics-informed GANs**: residual losses on the generator output for super-resolution of turbulence, downscaling.
- **Energy-based models / score networks**: useful when the unnormalised density is the natural object (statistical mechanics, molecular dynamics free energy).

## 9.6 Practical considerations

**Recipe: Training a diffusion-based PDE surrogate**

1. Choose noise schedule (variance-preserving, variance-exploding, EDM/Karras parameterisation).
2. Architectural backbone: U-Net for grids, transformers / FNO for heterogeneous data.
3. Condition on PDE coefficients, BCs, ICs, time index.

4. Loss: $\boldsymbol{v}$-prediction or $\epsilon$-prediction; physics residual penalty optional.
5. Sampling: deterministic ODE (DDIM, EDM) for speed, SDE for diversity; few-step distillation for inference.

**Pitfall: Generative samples that are not physical**

A diffusion model that scores well on pixel-MSE may still produce samples violating mass conservation, divergence-free constraints, or boundary conditions. Add hard projections onto the constraint manifold (divergence-free projection for incompressible flows) or physics-guided sampling.

# 10

# Uncertainty Quantification

*A prediction without uncertainty is a guess. A guess without uncertainty is malpractice.*

D. Saltzman

## 10.1 Sources of uncertainty

- **Aleatoric**: irreducible randomness in the data-generating process (turbulence, measurement noise).
- **Epistemic**: lack of knowledge about model parameters (reducible with more data).
- **Structural**: wrong model class — often the largest contributor in SciML and the hardest to quantify.

A trustworthy SciML pipeline must acknowledge all three.

## 10.2 Bayesian deep learning

### 10.2.1 Bayesian neural networks (BNNs)

Treat weights as random variables with prior $p(\theta)$ and posterior $p(\theta|\mathcal{D}) \propto p(\mathcal{D}|\theta)p(\theta)$. Approximations:

- **Variational inference**: mean-field, structured Gaussian, normalising-flow posteriors.
- **Laplace approximation**: $p(\theta|\mathcal{D}) \approx \mathcal{N}(\theta_{\mathrm{MAP}}, \boldsymbol{H}^{-1})$ via Hessian/GGN.
- **MCMC** (HMC, SGLD, SGNHT): asymptotically exact but costly.

### 10.2.2 Bayesian PINNs

B-PINN [41] samples from the posterior over $\theta$ given the residual + data likelihood, providing predictive intervals on PDE solutions. Used for noisy inverse problems and data-assimilation.

## 10.3 Deep ensembles

Train $K$ independent networks; average predictions and use the inter-model variance as uncertainty. Often the strongest baseline — simple, calibrated, trivially parallel. Drawback:

$K\times$ training and inference cost. Approximations: snapshot ensembles, BatchEnsemble, multi-input multi-output (MIMO).

## 10.4 Gaussian processes

The classical UQ workhorse. For a kernel $k(\boldsymbol{x}, \boldsymbol{x}')$ and observations with noise $\sigma^2$, posterior mean and variance are analytic. Limitations: $\mathcal{O}(N^3)$ training. Modern remedies:

- Sparse / inducing-point methods (SVGP, FITC).
- Stochastic variational GPs.
- Deep kernel learning: GP head on a neural feature map.
- Neural processes / Conditional NPs as amortised meta-GPs.

GPs are particularly natural surrogates for expensive forward simulators and for Bayesian optimisation of physical experiments.

## 10.5 Conformal prediction

Distribution-free, finite-sample-valid prediction sets:

$$\Pr(y_{\text{new}} \in C_\alpha(\boldsymbol{x}_{\text{new}})) \geq 1 - \alpha. \tag{10.1}$$

Build a non-conformity score on a calibration set; set the prediction interval as the $(1-\alpha)$-quantile. Variants for regression (CQR, conformalised quantile regression), time series (split-conformal with weighted exchangeability), and PDEs (conformalised neural operators) [42]. Crucially, conformal works *on top* of any predictor and gives marginal coverage with no distributional assumptions.

**Insight: Why conformal is special**

Other UQ methods are calibrated only if the model is well-specified (BNNs, GPs). Conformal coverage holds under exchangeability *regardless* of model correctness — making it ideal for hardening ML surrogates that one does not fully trust.

## 10.6 Probabilistic operators and probabilistic PINNs

- **Probabilistic FNO / DeepONet**: heteroscedastic outputs; ensemble FNOs; SDE-based latent operators.
- **Score-based UQ**: a diffusion prior + posterior sampling yields full predictive distributions for inverse PDE problems.
- **Monte Carlo dropout**: cheap approximation; usually overconfident.

## 10.7 Calibration and evaluation

- Reliability diagrams; Expected Calibration Error (ECE).
- Continuous Ranked Probability Score (CRPS) for ensembles.
- Negative log-likelihood (NLL) on held-out data.
- Coverage of prediction intervals; sharpness.
- Out-of-distribution (OOD) detection.

## 10.8 Active learning and experimental design

Couple UQ with sampling: select new query points to minimise expected posterior variance (BALD, expected-information-gain). For SciML:

- Active learning for force-field training (NequIP-AL).
- Bayesian optimisation for PDE-parameter calibration.
- Multi-fidelity Bayesian optimisation when both cheap and expensive simulators are available.

## 10.9 Practical guide

**Recipe: UQ stack for a SciML deliverable**

1. Train a deterministic baseline; compute residuals on held-out data.
2. Add an ensemble of $\geq 5$ models.
3. Apply conformal prediction on a calibration split.
4. If the model will be used for inverse problems, add a Bayesian or score-based posterior layer.
5. Report CRPS / NLL / coverage — never just RMSE.

**Pitfall: Confusing aleatoric with epistemic**

A heteroscedastic-output network estimates aleatoric noise. Adding ensembles or BNNs estimates epistemic. Plotting only the predictive variance without separating the two leads to incorrect inferences — e.g., believing more data will reduce noise that is intrinsic to the process.

# 11

# Multiscale and Multi-Physics SciML

*Nature does not separate scales for our convenience.*

Most real-world physical problems are inherently multiscale. Turbulence ranges from Kolmogorov microscales to integral scales; biology spans angstroms to organisms; the climate couples cloud microphysics to planetary circulation. Resolving every scale directly is computationally hopeless — which is why multiscale methods are a recurring battleground for SciML.

## 11.1 Classical multiscale methods

- **Homogenisation**: derive an effective equation by averaging over a microstructure.
- **HMM (heterogeneous multiscale method)**: alternate between macro and micro solvers.
- **Multigrid / multiresolution**: hierarchies of grids.
- **Wavelet methods**: scale-adaptive bases.
- **Quasi-continuum / coarse-graining**: bridging atomistic and continuum.

## 11.2 Where ML fits

- Learn microscale closures that feed into macroscale solvers (Chapter 7).
- Multi-resolution neural operators (U-FNO, multiscale transformer).
- Differentiable multigrid where smoothers are neural.
- Coarse-graining via diffusion models that preserve statistical ensembles of fine-scale fields.

## 11.3 Foundation models for multi-physics

A 2024–2026 trend: train one model on multiple PDE families, then fine-tune. Key examples (also revisited in Chapter 13):

- **Multiple Physics Pretraining (MPP)** [43].
- **DPOT** [44]: denoising pretrained operator transformer.
- **Poseidon** [21].

- **The Well** [45]: 15 TB of multi-physics simulation data for pretraining.

## 11.4 Coupled physics: examples

- **Fluid–structure interaction**: differentiable simulators plus surrogate closure models.
- **Climate–ocean–biogeochemistry**: NeuralGCM-style hybrid GCMs.
- **Plasma + EM fields**: gyrokinetic surrogates with neural closures.
- **Reactive flows**: neural surrogates for combustion chemistry coupled with CFD.
- **Astrophysical n-body + gas + radiation**: equivariant GNNs + radiative transfer surrogates.

## 11.5 Scale invariance and dimensional analysis

**Self-similar problems.** Many PDEs admit scaling symmetries (e.g. Navier–Stokes Reynolds similarity). Architectures that respect scale invariance generalise across regimes. Dimensional analysis (Buckingham $\pi$) reduces parameter spaces; embedding it as inductive bias narrows the search.

**Renormalisation-group ideas.** Recent work treats coarse-graining as a learned RG flow, particularly in statistical physics and lattice QFT, with normalising flows parameterising the renormalisation map.

## 11.6 Practical advice

**Recipe: Multiscale SciML workflow**

1. Identify scale separation; if present, design hybrid micro/macro schemes.
2. Decide whether ML replaces, accelerates, or closes a sub-step.
3. Validate at each scale before coupling — small errors at fine scales explode at coarse scales.
4. Quantify uncertainty across scales; conformal envelopes around coarse-grained predictions.
5. Where possible, use foundation operators pretrained on multi-scale corpora as starting points.

**Pitfall: Closure feedback loops**

A neural closure trained offline (one-shot) often fails when coupled back into a solver: the input distribution shifts as the solver evolves under the learned closure. Train *online* (with the solver in the loop) or use distribution-shift-robust losses.

# 12 Software, Datasets, and Benchmarks

## 12.1 Software ecosystems

### 12.1.1 Julia

- **SciML.jl** (DifferentialEquations.jl, Lux.jl, DiffEqFlux.jl, ModelingToolkit.jl, NeuralPDE.jl, NeuralOperators.jl): the premier ecosystem for differentiable scientific computing, universal differential equations, and symbolic-numeric modelling [46].
- **ModelingToolkit + ModelingToolkitNeuralNets**: symbolic PDE/DAE modelling with embedded neural blocks.
- **Optimization.jl, Lux.jl, Zygote.jl, Enzyme.jl**: forward, reverse, and source-to-source AD.

### 12.1.2 Python / JAX

- **JAX + Diffrax**: differentiable solvers (Tsit5, KenCarp, etc.) integrated with neural networks via Equinox/Flax/Haiku.
- **JAX-CFD, JAX-Fluids, JAX-MD, JAX-FEM**: fluid, molecular, and FEM solvers.
- **PhiFlow, Modulus**: NVIDIA's PINN/operator framework with multi-GPU support.
- **Neuraloperator (PyTorch)**: official FNO/TFNO implementation.
- **DeepXDE**: PINNs and DeepONets with multi-backend support.

### 12.1.3 Other

- **Firedrake / FEniCS / dolfin-adjoint**: differentiable FEM.
- **PyTorch Geometric, DGL**: GNN backbones.
- **e3nn, ESCNN**: equivariant network libraries.
- **TorchDiffEq, TorchSDE**: neural ODE/SDE in PyTorch.

## 12.2 Datasets and benchmarks

### 12.2.1 PDE benchmarks

- **PDEBench** [47]: advection, Burgers, Navier–Stokes (compressible/incompressible), Darcy, diffusion-reaction, shallow water; standardised metrics; widely cited.
- **The Well** [45]: 15 TB / 16 datasets across fluids, MHD, biology, granular media; built explicitly for foundation operators.

- **PDEArena**: clean PyTorch implementations of FNO, U-Net, DeepONet on Navier–Stokes / shallow water.
- **BubbleML**: multiphase flows.
- **CFDBench, AirfRANS**: aerodynamics.

### 12.2.2 Molecular / materials

- **MD17, rMD17, MD22**: gas-phase MD trajectories.
- **OC20, OC22**: catalysis with millions of DFT calculations.
- **QM9, QMugs**: quantum-chemistry property prediction.
- **Matbench, JARVIS**: materials property prediction.

### 12.2.3 Climate / weather

- **ERA5**: ECMWF reanalysis, the de-facto training corpus for ML weather models.
- **WeatherBench 2** [48]: standardised evaluation; benchmarks GraphCast, Pangu, FourCastNet, GenCast, NeuralGCM, etc.
- **ClimateBench, ClimSim**: climate emulation, subgrid parameterisation.

### 12.2.4 Astrophysics

- **CAMELS**: thousands of cosmological simulations with varying physics for ML training.
- **IllustrisTNG, EAGLE**: galaxy-formation simulation suites.
- **LSST, Euclid mocks**: synthetic survey data.

## 12.3 Reproducibility checklist

**Recipe: Publishing trustworthy SciML**

1. Code: open-source repository, fixed seeds, environment files.
2. Data: list source, version, preprocessing, splits.
3. Compute: report wall time, hardware, energy where possible.
4. Baselines: at least one classical solver and one prior NN method.
5. Metrics: error norm, calibration, OOD generalisation.
6. Ablations: which inductive biases matter.
7. Limitations: failure cases, regime of validity.

## 12.4 Hardware considerations

- GPUs (H100/H200/B200) for transformer/operator training; tensor cores reward large batched FFTs.
- TPUs (v5p) for very large foundation models (NeuralGCM, GraphCast).
- AMX / Apple Silicon for medium-scale prototyping (Mac mini M4 remains highly capable for FNO and PINN prototyping with MPS or PyTorch–Metal).
- Mixed precision (BF16/FP16) for inference; **always train PINNs in FP64** (Chapter 4).

# 13

# Emerging Trends (2024–2026)

*The cutting edge is where bibliographies become incomplete.*

This chapter surveys directions that are still moving fast and likely to shape the next phase of SciML. Citations indicate vintage 2024–2026.

## 13.1 Foundation models for physics

**Multiple Physics Pretraining (MPP) [43].** Train a single transformer on diverse physical fields (compressible NS, MHD, shear flows) using a unified channel-tokenised representation; fine-tune to new tasks with little data.

**Poseidon [21].** Multiscale operator transformer pretrained on a heterogeneous PDE mix with semi-group training; demonstrates strong transfer to unseen PDE families and downstream sample efficiency.

**DPOT [44].** Denoising pretrained operator transformer: combine masked autoencoding with operator regression to scale across PDE corpora.

**Other entrants.**

- **Aurora** (Microsoft, 2024): foundation atmospheric model for weather, air quality, ocean.
- **NeuralGCM [26]**: differentiable hybrid GCM with skill matching ECMWF on medium-range forecasts.
- **GenCast [39]**: diffusion ensemble forecaster, top of WeatherBench 2.
- **ORBIT (ORNL)**: 100-billion-parameter weather model on Frontier.
- **Pangu, GraphCast, FourCastNet**: 2022–2023 progenitors; the field's first "GPT moments".

**What is still missing.**

- Tokenisation of heterogeneous physical fields (different units, symmetries, dimensionality).
- Standardised evaluation across PDE families.
- Truly equivariant foundation models at scale.
- Long-horizon stability beyond a few hundred autoregressive steps.

- Open data + open weights: still patchy outside weather/climate.

## 13.2 Diffusion and flow-based generative simulators

The 2025 wave fuses generative modelling with PDE solving:

- **PDE-Refiner, PDE-Diffusion**: iterative denoising as a universal post-processor for operator outputs.
- **SimDiffPDE** [37]: plain transformer diffusion as a competitive PDE solver across a wide problem set.
- **PBFM** [40]: physics-based flow matching imposing PDE residuals into the flow-matching objective.
- **GenCast**: probabilistic ensemble forecasting at state-of-the-art skill.
- **Diffusion priors with physics-guided inference** [38]: train task-agnostic priors, condition at sampling time on PDE residuals — decoupling priors from PDE specifications.

## 13.3 Transformer operators

Transformers are the architecture backbone for the largest 2024–2026 SciML models:

- Attention naturally handles unstructured / variable-resolution inputs.
- Tokenisation of physical fields via patches, points, or learned tokens.
- Cross-attention between query coordinates and conditioning fields (DeepONet-style branches).
- Adaptive attention windows for multiscale problems.

Examples: Transolver, GAOT, OFormer, GNOT, MPP, DPOT, Poseidon.

## 13.4 Rethinking PINNs

A 2025 NeurIPS result [14] reframes PINN failure modes as *precision-induced optimisation pathologies* rather than fundamental landscape problems — L-BFGS' default convergence test fires too early in FP32. Switching to FP64 and using a stricter tolerance recovers vanilla-PINN convergence on benchmarks previously considered unsolvable. This is reshaping how the community discusses the limits of PINN training.

## 13.5 Equation discovery at scale

- **End-to-end transformer symbolic regression** (NeSymRes, SymbolicGPT, OmegaPRM-style search) is closing on PySR for benchmark problems.
- **LLM-guided equation discovery**: large language models proposing candidate symbolic forms, validated by numerical fitting (LaSR, MathConstellation).
- **Physics-aware program synthesis**: combining differentiable physics with neural-symbolic search for governing equations, hybrid ODEs, and stochastic models.

## 13.6 AI for science programmes

National-scale initiatives are reshaping funding and data:

- **DOE Foundation Models for Science** programmes (US): billions of CPU/GPU hours, multi-physics datasets, public foundation models.
- **EU AI Factories / Destination Earth**: digital twins of the planet with hybrid SciML.
- **UK AISI / Royal Society reports** on AI in science.
- **Industry labs**: DeepMind (GenCast, AlphaFold), NVIDIA Modulus, Microsoft (Aurora, DiG, MatterGen), Meta FAIR (OC, ESM).

## 13.7 Quantum-enhanced SciML

Quantum-classical hybrid solvers (variational quantum eigensolvers, quantum signal processing) intersect with neural surrogates for near-term quantum chemistry and lattice QFT. Currently early-stage but worth tracking.

## 13.8 Active learning, autonomous experimentation, and AI agents

LLM-powered agents that orchestrate simulation + experiment + ML modelling are an emerging mode. Examples: A-Lab (materials), ChemCrow, self-driving labs in materials science, agentic SciML pipelines that plan, run, and analyse simulations end-to-end.

## 13.9 Reinforcement learning for control of dynamical systems

- **Plasma control** (TCV, ITER digital twins).
- **Fluid control** (drag reduction, mixing).
- **Climate engineering scenarios** (still controversial).
- Differentiable physics + policy-gradient RL is converging into a unified control framework.

## 13.10 Where to bet next

**Insight: Five-year outlook**

- Foundation operators will become commodity; the moat moves to *data + downstream finetuning* pipelines.
- Probabilistic / generative surrogates will overtake deterministic ones for any high-stakes application requiring UQ.
- Equivariant + symplectic + transformer hybrids are likely architectural sweet spots.
- Hybrid mechanistic + neural models (UDEs, NeuralGCM) will outpace pure-NN approaches in safety-critical domains.
- LLM-mediated equation discovery and agentic SciML pipelines will democratise method design.

# 14

# Appendices

## Appendix A: Automatic Differentiation in 30 Lines

Automatic differentiation (AD) is the engine behind every method in this book. There are two primary modes:

**Forward mode.** Carries derivatives *with* the primal computation, using dual numbers $a + b\epsilon$ where $\epsilon^2 = 0$. Cost: $\mathcal{O}(\dim \text{input})$ times the primal. Best when input dimension is small.

**Reverse mode (backpropagation).** Records a tape during the forward pass; replays it backwards with accumulated cotangents. Cost: $\mathcal{O}(\dim \text{output})$. The dominant mode for ML where loss is scalar.

**Higher-order AD.** Required by PINNs (second derivatives in space, mixed time/space partials). Implementation must avoid graph-construction explosion; JAX's `jvp/vjp` composition and Diffrax/Equinox's higher-order support are best in class.

## Appendix B: Functional Analysis Cheat Sheet

- **Banach space**: complete normed vector space. Examples: $L^p$, $C^k$, $W^{k,p}$ (Sobolev).
- **Hilbert space**: Banach with inner product. Examples: $L^2$, $H^k = W^{k,2}$.
- **Sobolev embedding**: $W^{k,p}(\Omega) \hookrightarrow C^m(\bar{\Omega})$ when $kp > md$, $p > 1$.
- **Trace theorem**: boundary values of $H^1$ functions live in $H^{1/2}(\partial\Omega)$.
- **Compact operators**: limit (in operator norm) of finite-rank operators; spectra are discrete with 0 as only accumulation point.
- **Spectral theorem**: self-adjoint compact operators on a Hilbert space admit an orthonormal eigenbasis.

## Appendix C: Stochastic Calculus in Half a Page

**Itô's lemma.** For $\mathrm{d}\boldsymbol{X}_t = \boldsymbol{f}\mathrm{d}t + \boldsymbol{\sigma}\mathrm{d}\boldsymbol{W}_t$ and $g \in C^{2,1}$,

$$\mathrm{d}g(\boldsymbol{X}_t, t) = \Big(\partial_t g + \boldsymbol{f} \cdot \nabla g + \tfrac{1}{2}\operatorname{Tr}(\boldsymbol{\sigma}\boldsymbol{\sigma}^\mathsf{T}\nabla^2 g)\Big)\,\mathrm{d}t + (\nabla g)^\mathsf{T}\boldsymbol{\sigma}\mathrm{d}\boldsymbol{W}_t.$$

**Fokker–Planck (forward Kolmogorov).** Density $p(\boldsymbol{x}, t)$ obeys

$$\partial_t p = -\nabla \cdot (\boldsymbol{f}p) + \tfrac{1}{2}\nabla\nabla : (\boldsymbol{\sigma}\boldsymbol{\sigma}^\mathsf{T}p).$$

**Anderson's reverse-time SDE.** The reverse-time process associated with the forward SDE above is

$$\mathrm{d}\bar{\boldsymbol{X}}_t = (\boldsymbol{f} - \boldsymbol{\sigma}\boldsymbol{\sigma}^\top \nabla \log p_t)\mathrm{d}t + \boldsymbol{\sigma}\mathrm{d}\bar{\boldsymbol{W}}_t,$$

the foundation of score-based generative models (Chapter 9).

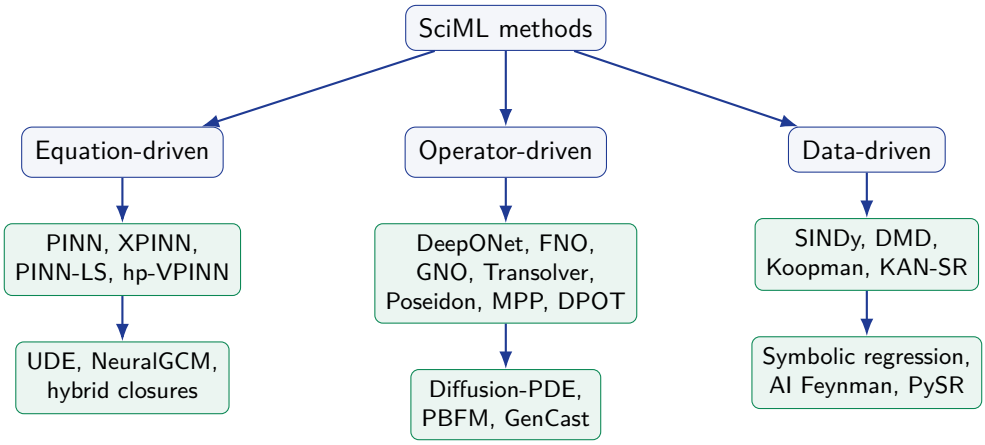
## Appendix D: Reading List by Topic

- **Dynamical systems**: Strogatz (*Nonlinear Dynamics and Chaos*); Guckenheimer & Holmes; Hirsch–Smale–Devaney.
- **Numerical PDEs**: LeVeque (*FV Methods*); Quarteroni; Trefethen (*Spectral Methods in MATLAB*).
- **SciML overview**: Karniadakis et al. (*Nature Reviews Physics*, 2021); Brunton & Kutz (*Data-Driven Science*); Cuomo et al. (2022 PINN survey).
- **Operator learning**: Kovachki et al. (2023 review); Lu et al. (DeepONet); Li et al. (FNO).
- **Koopman/SINDy**: Mezić (*Annu. Rev. Fluid Mech.*); Brunton, Budišić, Kaiser & Kutz (Koopman review 2022).
- **Generative SciML**: Song & Ermon SDE paper; Lipman flow matching; recent NeurIPS/ICLR proceedings on diffusion for PDEs.
- **Equivariant DL**: Bronstein et al. *Geometric Deep Learning*; Cohen & Welling; Batzner et al. (NequIP).
- **UQ**: Gal (BNNs thesis); Vovk *Algorithmic Learning in a Random World* (conformal); Rasmussen & Williams (GPs).

## Appendix E: Glossary of Key Acronyms

- **AD** – Automatic Differentiation
- **AFNO** – Adaptive FNO
- **BNN** – Bayesian Neural Network
- **CFL** – Courant–Friedrichs–Lewy
- **CNF** – Continuous Normalising Flow
- **DMD** – Dynamic Mode Decomposition
- **DPOT** – Denoising Pretrained Operator Transformer
- **EGNN** – E(n)-Equivariant GNN
- **FNO** – Fourier Neural Operator
- **GAOT** – Geometry-Aware Operator Transformer
- **GCM** – General Circulation Model
- **GNO** – Graph Neural Operator
- **HMM** – Heterogeneous Multiscale Method
- **HNN/LNN** – Hamiltonian/Lagrangian NN
- **KAN** – Kolmogorov–Arnold Network
- **MGN** – MeshGraphNet
- **MPP** – Multiple Physics Pretraining
- **NTK** – Neural Tangent Kernel
- **ODE/PDE/SDE** – Ordinary / Partial / Stochastic DE
- **PBFM** – Physics-Based Flow Matching
- **PINN** – Physics-Informed Neural Network
- **PINO** – Physics-Informed Neural Operator
- **POD** – Proper Orthogonal Decomposition
- **ROM** – Reduced-Order Model
- **SFNO** – Spherical FNO
- **SINDy** – Sparse Identification of Nonlinear Dynamics
- **TFN** – Tensor-Field Network
- **UDE** – Universal Differential Equation
- **UQ** – Uncertainty Quantification
- **XPINN/cPINN** – Domain-decomposition PINN variants

## Appendix F: Method Taxonomy at a Glance



# References & Further Reading


[1] R. T. Q. Chen, Y. Rubanova, J. Bettencourt, and D. Duvenaud. "Neural Ordinary Differential Equations". In: *Advances in Neural Information Processing Systems (NeurIPS)* (2018). URL: https://arxiv.org/abs/1806.07366.

[2] E. Dupont, A. Doucet, and Y. W. Teh. "Augmented Neural ODEs". In: *Advances in Neural Information Processing Systems (NeurIPS)* (2019). URL: https://arxiv.org/abs/1904.01681.

[3] Y. Rubanova, R. T. Q. Chen, and D. Duvenaud. "Latent ODEs for Irregularly-Sampled Time Series". In: *NeurIPS* (2019). URL: https://arxiv.org/abs/1907.03907.

[4] P. Kidger, J. Morrill, J. Foster, and T. Lyons. "Neural Controlled Differential Equations for Irregular Time Series". In: *NeurIPS* (2020). URL: https://arxiv.org/abs/2005.08926.

[5] S. Greydanus, M. Dzamba, and J. Yosinski. "Hamiltonian Neural Networks". In: *NeurIPS* (2019). URL: https://arxiv.org/abs/1906.01563.

[6] M. Cranmer, S. Greydanus, S. Hoyer, P. Battaglia, D. Spergel, and S. Ho. "Lagrangian Neural Networks". In: *ICLR Workshop* (2020). URL: https://arxiv.org/abs/2003.04630.

[7] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel, and M. Le. "Flow Matching for Generative Modeling". In: *ICLR* (2023). URL: https://arxiv.org/abs/2210.02747.

[8] M. S. Albergo, N. M. Boffi, and E. Vanden-Eijnden. "Stochastic Interpolants: A Unifying Framework for Flows and Diffusions". In: *arXiv preprint arXiv:2303.08797* (2023). URL: https://arxiv.org/abs/2303.08797.

[9] M. Raissi, P. Perdikaris, and G. E. Karniadakis. "Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations". In: *Journal of Computational Physics* 378 (2019), pp. 686–707. URL: https://doi.org/10.1016/j.jcp.2018.10.045.

[10] A. D. Jagtap and G. E. Karniadakis. "Extended Physics-Informed Neural Networks (XPINNs): A Generalized Space-Time Domain Decomposition Based Deep Learning Framework". In: *Communications in Computational Physics* (2020).

[11] S. Wang, S. Sankaran, and P. Perdikaris. "Respecting Causality is All You Need for Training Physics-Informed Neural Networks". In: *arXiv preprint arXiv:2203.07404* (2022). URL: https://arxiv.org/abs/2203.07404.

[12] S. Wang, X. Yu, and P. Perdikaris. "When and why PINNs fail to train: A neural tangent kernel perspective". In: *Journal of Computational Physics* (2022). URL: https://doi.org/10.1016/j.jcp.2021.110768.

[13] L. McClenny and U. Braga-Neto. "Self-adaptive physics-informed neural networks". In: *Journal of Computational Physics* (2023).

[14] V. Authors. "Rethinking Failure Modes in Physics-Informed Neural Networks". In: *NeurIPS* (2025). URL: https://arxiv.org/abs/2505.10949.

[15] T. De Ryck and S. Mishra. "Generic bounds on the approximation error for physics-informed (and) operator learning". In: *Acta Numerica* (2024).

[16] L. Lu, P. Jin, G. Pang, Z. Zhang, and G. E. Karniadakis. "Learning nonlinear operators via DeepONet based on the universal approximation theorem of operators". In: *Nature Machine Intelligence* 3.3 (2021), pp. 218–229. URL: https://www.nature.com/articles/s42256-021-00302-5.

[17] Z. Li et al. "Fourier Neural Operator for Parametric Partial Differential Equations". In: *ICLR* (2021). URL: https://arxiv.org/abs/2010.08895.

[18] B. Bonev et al. "Spherical Fourier Neural Operators: Learning Stable Dynamics on the Sphere". In: *ICML* (2023). URL: https://arxiv.org/abs/2306.03838.

[19] Z. Li et al. "Neural Operator: Graph Kernel Network for Partial Differential Equations". In: *arXiv preprint arXiv:2003.03485* (2020).

[20] J. Brandstetter, D. Worrall, and M. Welling. "Message Passing Neural PDE Solvers". In: *ICLR* (2022). URL: https://arxiv.org/abs/2202.03376.

[21] M. Herde et al. "Poseidon: Efficient Foundation Models for PDEs". In: *NeurIPS* (2024). URL: https://arxiv.org/abs/2405.19101.

[22] N. Kovachki et al. "Neural Operator: Learning Maps Between Function Spaces". In: *Journal of Machine Learning Research* (2023). URL: https://www.jmlr.org/papers/v24/21-1524.html.

[23] B. Lusch, J. N. Kutz, and S. L. Brunton. "Deep learning for universal linear embeddings of nonlinear dynamics". In: *Nature Communications* (2018). URL: https://www.nature.com/articles/s41467-018-07210-0.

[24] S. L. Brunton, J. L. Proctor, and J. N. Kutz. "Discovering governing equations from data by sparse identification of nonlinear dynamical systems". In: *Proceedings of the National Academy of Sciences* (2016). URL: https://doi.org/10.1073/pnas.1517384113.

[25] C. Rackauckas et al. "Universal Differential Equations for Scientific Machine Learning". In: *arXiv preprint arXiv:2001.04385* (2020). URL: https://arxiv.org/abs/2001.04385.

[26] D. Kochkov, J. Yuval, et al. "Neural general circulation models for weather and climate". In: *Nature* (2024). URL: https://www.nature.com/articles/s41586-024-07744-y.

[27] Z. Liu et al. "KAN: Kolmogorov-Arnold Networks". In: *arXiv preprint arXiv:2404.19756* (2024). URL: https://arxiv.org/abs/2404.19756.

[28] N. Thomas et al. "Tensor field networks: Rotation- and translation-equivariant neural networks for 3D point clouds". In: *arXiv preprint arXiv:1802.08219* (2018).

[29] F. Fuchs, D. Worrall, V. Fischer, and M. Welling. "SE(3)-Transformers: 3D Roto-Translation Equivariant Attention Networks". In: *NeurIPS* (2020).

[30] S. Batzner et al. "E(3)-equivariant graph neural networks for data-efficient and accurate interatomic potentials". In: *Nature Communications* (2022).

[31] V. G. Satorras, E. Hoogeboom, and M. Welling. "E(n) Equivariant Graph Neural Networks". In: *ICML* (2021). URL: https://arxiv.org/abs/2102.09844.

[32] T. Pfaff, M. Fortunato, A. Sanchez-Gonzalez, and P. W. Battaglia. "Learning Mesh-Based Simulation with Graph Networks". In: *ICLR* (2021). URL: https://arxiv.org/abs/2010.03409.

[33] A. Sanchez-Gonzalez, J. Godwin, T. Pfaff, R. Ying, J. Leskovec, and P. W. Battaglia. "Learning to Simulate Complex Physics with Graph Networks". In: *ICML* (2020).

[34] Y. Song, J. Sohl-Dickstein, D. P. Kingma, A. Kumar, S. Ermon, and B. Poole. "Score-Based Generative Modeling through Stochastic Differential Equations". In: *ICLR* (2021). URL: https://arxiv.org/abs/2011.13456.

[35] P. Lippe, B. S. Veeling, P. Perdikaris, R. E. Turner, and J. Brandstetter. "PDE-Refiner: Achieving Accurate Long Rollouts with Neural PDE Solvers". In: *NeurIPS* (2023). URL: https://arxiv.org/abs/2308.05732.

[36] S. R. Cachay, B. Zhao, H. Joren, and R. Yu. "DYffusion: A Dynamics-informed Diffusion Model for Spatiotemporal Forecasting". In: *NeurIPS* (2023).

[37] Anonymous. "Simple Diffusion Baselines for Solving Partial Differential Equations". In: *ICLR* (2025). URL: https://openreview.net/forum?id=JQV9gH55Az.

[38] V. Authors. "Diffusion models with physics-guided inference for solving partial differential equations". In: *arXiv preprint arXiv:2604.01242* (2026). URL: https://arxiv.org/abs/2604.01242.

[39] I. Price et al. "Probabilistic weather forecasting with machine learning (GenCast)". In: *Nature* (2024). URL: https://www.nature.com/articles/s41586-024-08252-9.

[40] V. Authors. "Flow Matching Meets PDEs: A Unified Framework for Physics-Constrained Generation". In: *arXiv preprint arXiv:2506.08604* (2025). URL: https://arxiv.org/abs/2506.08604.

[41] L. Yang, X. Meng, and G. E. Karniadakis. "B-PINNs: Bayesian Physics-Informed Neural Networks for Forward and Inverse PDE Problems with Noisy Data". In: *Journal of Computational Physics* (2021).

[42] Z. Ma, K. Azizzadenesheli, and A. Anandkumar. "Calibrated Uncertainty Quantification for Operator Learning via Conformal Prediction". In: *arXiv preprint arXiv:2402.01960* (2024). URL: https://arxiv.org/abs/2402.01960.

[43] M. McCabe, B. R.-S. Blancard, L. Parker, R. Ohana, M. Cranmer, A. Bietti, et al. "Multiple Physics Pretraining for Physical Surrogate Models". In: *NeurIPS* (2024). URL: https://arxiv.org/abs/2310.02994.

[44] Z. Hao et al. "DPOT: Auto-Regressive Denoising Operator Transformer for Large-Scale PDE Pre-Training". In: *ICML* (2024). URL: https://arxiv.org/abs/2403.03542.

[45] R. Ohana, M. McCabe, L. Meyer, R. Morel, F. Agocs, et al. "The Well: a Large-Scale Collection of Diverse Physics Simulations for Machine Learning". In: *NeurIPS Datasets and Benchmarks* (2024). URL: https://arxiv.org/abs/2412.00568.

[46] C. Rackauckas and Q. Nie. "DifferentialEquations.jl – A Performant and Feature-Rich Ecosystem for Solving Differential Equations in Julia". In: *Journal of Open Research Software* (2017).

[47] M. Takamoto et al. "PDEBench: An Extensive Benchmark for Scientific Machine Learning". In: *NeurIPS Datasets and Benchmarks* (2022). URL: https://arxiv.org/abs/2210.07182.

[48] S. Rasp et al. "WeatherBench 2: A Benchmark for the Next Generation of Data-Driven Global Weather Models". In: *Journal of Advances in Modeling Earth Systems* (2024).